UNIVERSIDAD COOPERATIVA DE COLOMBIA

Sede Montería

Facultad de Ingeniería

Programa de Ingeniería de Sistemas

Electiva II – Minería de Datos

Ecosistema Analítico End-to-End para Scouting Deportivo: Clasificación de Jugadores Élite con Random Forest y Despliegue mediante API REST

Informe ejecutivo del proyecto final – Momento 3

Presentado por:

David Cuello

Docente:

Magíster Royer Mejía Villalba

Montería, Córdoba

Noviembre de 2026

Tabla de contenido

1. Introducción y objetivos 4

1.1. Objetivo general 4

1.2. Objetivos específicos 4

1.3. Justificación bajo la metodología CRISP-DM 5

2. Marco teórico y glosario 7

2.1. Fase 1 – Ingesta, preprocesamiento e ingeniería de características 7

2.2. Fase 2 – Modelado, diagnóstico y optimización 9

2.3. Fase 3 – Evaluación avanzada y despliegue 11

2.4. Conceptos complementarios de despliegue y monitoreo 13

3. Análisis de resultados del código y experimentos 15

3.1. Descripción de los datos y del preprocesamiento 15

3.2. Paradoja de la exactitud 17

3.3. Diagnóstico de overfitting: árbol no restringido vs. árbol podado 18

3.4. Evaluación del ensamble y curva ROC-AUC 20

3.5. Importancia de variables: ¿qué variables pesan más y por qué? 23

4. Arquitectura de despliegue 26

4.1. Exportación del modelo 26

4.2. Contrato JSON de la API 27

4.3. API en ejecución 28

4.4. Aplicativo web respondiendo a jugadores nuevos 31

5. Conclusiones y recomendaciones de negocio 35

5.1. Conclusiones 35

5.2. Cómo apoya el sistema la toma de decisiones en scouting 35

5.3. Limitaciones....36
5.4. Recomendaciones y trabajo futuro....36
6. Referencias....38
7. Anexos....40
Anexo A. Estructura de carpetas del proyecto....40
Anexo B. Instrucciones de ejecución....40

1. Introducción y objetivos

En el fútbol profesional, el departamento de *scouting* (ojeo) debe filtrar miles de perfiles para encontrar a los pocos jugadores que justifican una inversión. Cada visoría presencial cuesta viajes, tiempo de analistas y oportunidades perdidas, y un fichaje equivocado compromete presupuestos de varios millones de euros. Los clubes con recursos limitados necesitan, por tanto, herramientas que prioricen candidatos de forma objetiva antes de destinar recursos humanos a observarlos.

Este proyecto construye un **ecosistema analítico de extremo a extremo** sobre el conjunto de datos de Transfermarkt almacenado en MySQL (base `mineria_datos`, tabla `players`, 50 149 jugadores). El sistema extrae los datos con un conector propio, los depura, entrena y optimiza un clasificador *Random Forest* que estima si un jugador pertenece al **25 % más valioso del mercado (clase Élite, valor ≥ € 700 000)**, serializa el modelo con Joblib, lo expone como una **API REST** (FastAPI) y lo pone en manos del usuario final mediante un **aplicativo web** interactivo. Así, el modelo deja de vivir en un *notebook* y se convierte en un servicio que cualquier scout puede consultar.

1.1. Objetivo general

Desarrollar y desplegar un sistema de clasificación de jugadores de fútbol en las categorías Élite y Estándar a partir de variables físicas y demográficas (edad, estatura y posición), siguiendo la metodología CRISP-DM, desde la extracción de datos relacionales en MySQL hasta su consumo mediante una API REST y una interfaz web.

1.2. Objetivos específicos

- Extraer los datos de jugadores desde MySQL con el conector `MySQLConnector` (`connection2.py`) y tratar los valores faltantes mediante imputación por mediana agrupada por posición.
- Construir un target binario por percentil (Top 25 %) y preparar las variables con One-Hot Encoding y partición estratificada.

- Diagnosticar el sobreajuste comparando un árbol de decisión sin restricciones contra uno con poda previa, y evidenciar la paradoja de la exactitud en un problema desbalanceado.
- Corregir el desbalance con `class_weight='balanced'` y optimizar un `RandomForestClassifier` con `GridSearchCV` y validación cruzada K-Fold estratificada.
- Evaluar los modelos con curvas ROC y AUC, e interpretar la importancia de variables (Gini y permutación).
- Serializar el paquete del modelo con Joblib, desplegarlo en una API REST con FastAPI y Pydantic, y consumirlo desde un aplicativo web con formulario, `fetch` y barra de progreso.

1.3. Justificación bajo la metodología CRISP-DM

CRISP-DM (*Cross-Industry Standard Process for Data Mining*; Chapman et al., 2000) es el estándar de facto para proyectos de minería de datos: organiza el trabajo en seis fases iterativas que conectan la necesidad del negocio con un modelo en producción. Se eligió porque el proyecto no termina en una métrica, sino en un servicio desplegado, y CRISP-DM es la única de las metodologías vistas en el curso que incluye explícitamente la fase de **Despliegue**. La Tabla 1 y la Figura 1 resumen cómo se cubrió cada fase.

Tabla 1

Correspondencia entre las fases de CRISP-DM y el trabajo realizado

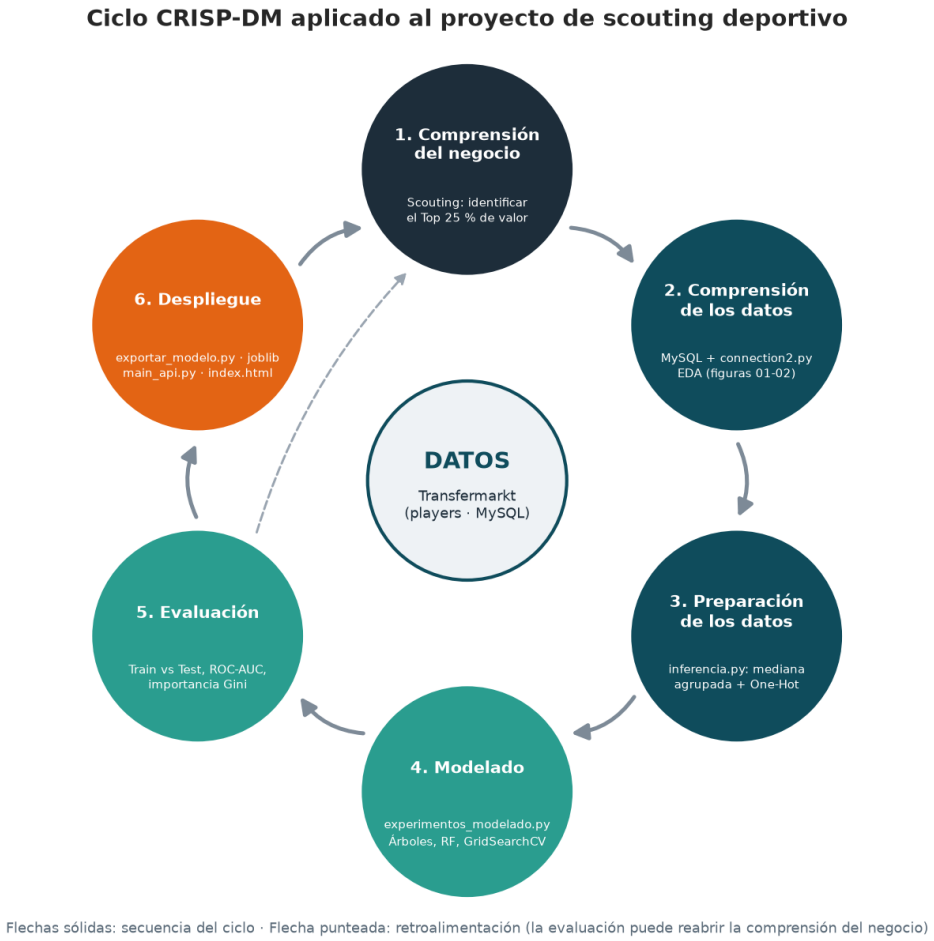
Fase CRISP-DM	Aplicación en el proyecto	Artefacto
1. Comprensión del negocio	Problema: priorizar candidatos con alto valor de mercado para reducir costos de visoría. Éxito: detectar la mayoría de jugadores Élite (recall alto) con buena capacidad de discriminación (AUC).	Este informe, §1
2. Comprensión de los datos	Consulta SQL a `players`: 41 495 registros válidos, 2 183 estaturas nulas, 5 posiciones, umbral Q3 de € 700 000.	`connection2.py`, figuras 2 y 3
3. Preparación de los datos	Imputación por mediana agrupada, target binario P75, One-Hot Encoding y split estratificado 80/20 sin fuga de datos.	`src/preprocessing/inferencia.py`
4. Modelado	Árbol libre, árbol podado, árbol balanceado y Random Forest optimizado con GridSearchCV (72 combinaciones × 5 folds).	`experimentos_modelado.py`
5. Evaluación	Accuracy Train/Test, Precision, Recall, F1, matrices de confusión, ROC-AUC e importancia de variables.	`reports/resultados/`

6. Despliegue	Paquete Joblib, API REST con FastAPI y aplicativo web.	`exportar_modelo.py`, `main_api.py`, `index.html`

Nota. Elaboración propia.

Figura 1

Ciclo CRISP-DM aplicado al proyecto y scripts que cubren cada fase



Nota. Elaboración propia con base en Chapman et al. (2000).

2. Marco teórico y glosario

Esta sección define con rigor los catorce conceptos exigidos por la guía de la actividad. Para cada uno se presentan su definición, su formulación cuando aplica, la razón por la que se usa y **cómo se aplicó concretamente en el código** del proyecto. Al final se incluyen los conceptos de despliegue y monitoreo de la última clase.

2.1. Fase 1 – Ingesta, preprocesamiento e ingeniería de características

2.1.1. Imputación por mediana agrupada

Definición. Técnica de tratamiento de valores faltantes que reemplaza cada nulo por la mediana del subgrupo al que pertenece el registro (aquí, la posición), en lugar de un único valor global.

$$\hat{x}_i = mediana\{ x_j : posición_j = posición_i \}$$

¿Por qué se usa? La mediana es robusta frente a valores atípicos (a diferencia de la media) y el agrupamiento respeta la estructura real de los datos: un portero no mide lo mismo que un mediocampista (Figura 3). Imputar con la mediana global sesgaría a los porteros hacia abajo y a los mediocampistas hacia arriba.

Aplicación en el proyecto. En `src/preprocessing/inferencia.py`, `calcular_medianas()` aprende `df.groupby('posicion')['estatura_cm'].median()` y `imputar()` aplica `fillna(df['posicion'].map(medianas_estatura))`, con la mediana global como respaldo; la edad se imputa con la mediana global. Las medianas se guardan en el paquete `.joblib` (`medianas_estatura`, `mediana_global_estatura`, `mediana_global_edad`).

2.1.2. Target binario por percentil (Top 25 %)

Definición. Transformación de una variable continua (valor de mercado) en una etiqueta categórica binaria mediante un umbral estadístico: el tercer cuartil (Q3 o percentil 75).

$$y_i = 1 \text{ si } valor_i \geq P_{75}(valor); \quad y_i = 0 \text{ en otro caso}$$

¿Por qué se usa? Convierte un problema de regresión con una distribución muy asimétrica (pocos jugadores valen decenas de millones) en una decisión de negocio clara: ¿vale la pena

seguir a este jugador? El percentil hace que el umbral sea relativo al mercado y no una cifra arbitraria.

Aplicación en el proyecto. `umbral_elite = df['valor_mercado'].quantile(0.75)` (€ 700 000) y `df['es_elite'] = (df['valor_mercado'] >= umbral_elite).astype(int)` en `exportar_modelo.py` y `experimentos_modelado.py`. El umbral se guarda como `umbral_elite_eur`.

2.1.3. One-Hot Encoding (get_dummies)

Definición. Conversión de una variable categórica nominal en un conjunto de columnas binarias (variables *dummy*), una por categoría, que valen 1 si el registro pertenece a esa categoría y 0 en otro caso.

$$posición \in \{A, D, G, M, X\} \rightarrow (x_D, x_G, x_M, x_X) \in \{0,1\}^4$$

¿Por qué se usa? Los algoritmos de scikit-learn requieren entradas numéricas. Asignar números arbitrarios (Delantero = 1, Defensa = 2...) inventaría un orden inexistente. Con `drop_first=True` se elimina una categoría de referencia (Delantero) para evitar redundancia (colinealidad perfecta o *trampa de las dummies*).

Aplicación en el proyecto. `pd.get_dummies(df[['posicion']], drop_first=True, dtype=int)` genera `posicion_Defender`, `posicion_Goalkeeper`, `posicion_Midfield`, `posicion_Missing`. En inferencia se usa `get_dummies` sin `drop_first` seguido de `reindex(columns=columnas_X, fill_value=0)` para obtener exactamente las mismas columnas del entrenamiento (ver §2.4.2).

2.1.4. Muestreo estratificado (stratify)

Definición. Técnica de partición Train/Test que conserva en cada subconjunto la misma proporción de clases que existe en la población original.

$$P(y=1 \mid Train) \approx P(y=1 \mid Test) \approx P(y=1)$$

¿Por qué se usa? Con clases desbalanceadas, una partición aleatoria simple podría dejar al conjunto de prueba con muy pocos jugadores Élite y producir métricas inestables o engañosas.

Aplicación en el proyecto. `train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)` en `experimentos_modelado.py`. Resultado: Élite = 26.696 % en Train y 26.690 % en Test. `StratifiedKFold` aplica el mismo principio dentro de la validación cruzada.

2.2. Fase 2 – Modelado, diagnóstico y optimización

2.2.1. Sobreajuste (overfitting)

Definición. Fenómeno en el que un modelo memoriza el ruido y los detalles particulares del conjunto de entrenamiento, de modo que su desempeño en datos nuevos es sensiblemente inferior.

$$Brecha = Accuracy_{Train} - Accuracy_{Test}$$

¿Por qué se usa? Un modelo sobreajustado da una falsa sensación de calidad y falla en producción. La brecha Train − Test es su síntoma principal.

Aplicación en el proyecto. `DecisionTreeClassifier(random_state=42)` sin restricciones creció hasta profundidad 25 con 2 693 hojas y mostró una brecha de +0.0418 (§3.3).

2.2.2. Poda previa (pre-pruning)

Definición. Estrategia de regularización que detiene el crecimiento del árbol fijando límites estructurales antes de entrenarlo: profundidad máxima (`max_depth`) y número mínimo de muestras por hoja (`min_samples_leaf`).

¿Por qué se usa? Impide que el árbol cree reglas para casos aislados; cada decisión debe estar respaldada por un número mínimo de jugadores, lo que mejora la generalización y la interpretabilidad.

Aplicación en el proyecto. `DecisionTreeClassifier(max_depth=4, min_samples_leaf=20)`: el árbol pasó de 2 693 a 16 hojas y la brecha cayó a prácticamente cero. En el Random Forest se exploraron `max_depth` y `min_samples_leaf` dentro de la malla.

2.2.3. Paradoja de la exactitud (accuracy paradox)

Definición. Situación propia de datos desbalanceados en la que un modelo obtiene una exactitud alta simplemente prediciendo siempre la clase mayoritaria, sin ninguna capacidad real de detectar la clase minoritaria.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

¿Por qué se usa? Demuestra que la exactitud no es una métrica adecuada cuando la clase de interés es minoritaria; deben usarse Recall, Precision, F1 y AUC.

Aplicación en el proyecto. `DummyClassifier(strategy='most_frequent')` obtuvo accuracy = 0.7331 con Recall Élite = 0.0. El árbol podado sin balanceo cayó en la misma trampa (§3.2).

2.2.4. Penalización por pesos de costo (class_weight='balanced')

Definición. Técnica algorítmica que modifica la función de pérdida para que equivocarse con la clase minoritaria cueste más que equivocarse con la mayoritaria. Cada clase c recibe un peso inversamente proporcional a su frecuencia:

$$w_c = n / (k \cdot n_c)$$

¿Por qué se usa? Donde n es el número de muestras, k el número de clases y n_c las muestras de la clase c. Con 26.7 % de Élite, $w_{Élite} \approx 1.87$ y $w_{Estándar} \approx 0.68$: un error sobre un jugador Élite pesa casi tres veces más. No crea datos sintéticos (a diferencia de SMOTE).

Aplicación en el proyecto. `class_weight='balanced'` en el árbol podado balanceado y en `RandomForestClassifier` de `experimentos_modelado.py` y `exportar_modelo.py`.

2.2.5. Random Forest y ensamble (Bagging, Bootstrap y Feature Subspacing)

Definición. Algoritmo de aprendizaje por ensamble (Breiman, 2001) que entrena muchos árboles de decisión independientes y combina sus votos. Cada árbol se entrena con una muestra *bootstrap* (muestreo con reemplazo del conjunto de entrenamiento) y, en cada división, solo considera un subconjunto aleatorio de variables (*feature subspacing*, parámetro `max_features`).

$$\hat{p}(Élite \mid x) = (1/B) \cdot \Sigma_{b=1..B}\, \hat{p}_b(Élite \mid x)$$

¿Por qué se usa? *Bagging* (bootstrap aggregating) reduce la varianza: los errores de árboles poco correlacionados se compensan al promediar. El subespacio de variables descorrelaciona aún más los árboles. El resultado es un modelo más estable y con mejor generalización que un árbol único.

Aplicación en el proyecto. `RandomForestClassifier(class_weight='balanced', random_state=42, n_jobs=-1)` con los mejores hiperparámetros: `n_estimators=300`, `max_depth=8`, `min_samples_leaf=5`, `max_features='sqrt'`.

2.2.6. Búsqueda combinatoria con GridSearchCV y K-Fold Cross-Validation

Definición. Exploración exhaustiva de todas las combinaciones de una malla de hiperparámetros, evaluando cada una con validación cruzada K-Fold: el entrenamiento se divide en K partes, se entrena con K − 1 y se valida con la restante, rotando K veces y promediando el resultado.

$$Score_{CV} = (1/K) \cdot \Sigma_{k=1..K} AUC_k$$

¿Por qué se usa? Evita elegir hiperparámetros por ensayo y error o ajustándolos al conjunto de prueba (lo que sería otra forma de fuga de información). El promedio de K evaluaciones es una estimación más fiable que una sola partición.

Aplicación en el proyecto. `GridSearchCV(..., cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42), scoring={'roc_auc', 'f1'}, refit='roc_auc')` sobre 72 combinaciones (360 entrenamientos, 5.6 minutos).

2.3. Fase 3 – Evaluación avanzada y despliegue

2.3.1. Curva ROC y métrica AUC

Definición. La curva ROC (*Receiver Operating Characteristic*) grafica la Tasa de Verdaderos Positivos frente a la Tasa de Falsos Positivos para todos los umbrales de decisión posibles. El AUC (*Area Under the Curve*) resume la curva en un número entre 0 y 1 (Fawcett, 2006).

$$TPR = TP / (TP + FN) \qquad FPR = FP / (FP + TN)$$

¿Por qué se usa? A diferencia de la exactitud, no depende de un umbral fijo ni del desbalance. Su lectura probabilística es intuitiva: el AUC es la probabilidad de que el modelo asigne una puntuación más alta a un jugador Élite elegido al azar que a uno Estándar elegido al azar. AUC = 0.5 equivale al azar (diagonal) y AUC = 1.0 a una clasificación perfecta.

Aplicación en el proyecto. `roc_curve(y_test, predict_proba(X_test)[:, 1])` y `roc_auc_score` en `experimentos_modelado.py`; también es la métrica `scoring='roc_auc'` que optimiza el GridSearchCV (Figura 6).

2.3.2. Importancia de variables (Gini importance)

Definición. Medida de la contribución de cada variable a la reducción de impureza de Gini, sumada sobre todas las divisiones en que interviene y promediada entre los árboles del bosque. La impureza de Gini de un nodo con proporciones de clase p_i es:

$$Gini = 1 - \Sigma_i p_i^2$$

¿Por qué se usa? Permite explicar el modelo al negocio: qué características impulsan la predicción. Como la importancia Gini tiende a favorecer variables continuas con muchos puntos de corte (Strobl et al., 2007), se contrastó con la importancia por permutación, que mide cuánto cae el AUC en Test al desordenar cada variable.

Aplicación en el proyecto. `rf.feature_importances_` (Figura 7) y `permutation_importance(rf, X_test, y_test, scoring='roc_auc', n_repeats=10)` (Figura 8).

2.3.3. Serialización con Joblib (Joblib vs. Pickle)

Definición. Proceso de convertir objetos de Python que viven en memoria RAM en un archivo binario en disco para reutilizarlos después en otro proceso o servidor. Pickle es el módulo nativo de Python para objetos generales; Joblib está optimizado para objetos de scikit-learn que contienen grandes arreglos de NumPy, por lo que es más rápido y eficiente en espacio para modelos como un Random Forest.

¿Por qué se usa? Sin serialización, el modelo, las reglas de sus árboles y las medianas de imputación se pierden al cerrar el proceso de Python; habría que reentrenar cada vez. Serializar separa el entrenamiento (costoso, offline) de la inferencia (rápida, online).

Aplicación en el proyecto. `joblib.dump(paquete_despliegue, 'models/modelo_scouting_rf.joblib')` en `exportar_modelo.py` y `joblib.load(...)` una sola vez en el `lifespan` de `main_api.py`.

2.3.4. API REST y arquitectura cliente-servidor

Definición. Una API REST (*Representational State Transfer*; Fielding, 2000) es un servicio web que expone recursos mediante URL y verbos HTTP (GET, POST…) e intercambia representaciones en JSON. En la arquitectura cliente-servidor, el servidor (la API) concentra el modelo y la lógica, y uno o varios clientes (el navegador, otra aplicación) le envían peticiones.

¿Por qué se usa? Desacopla el modelo de quien lo usa: el aplicativo web, una app móvil o un sistema del club pueden consumir la misma predicción sin conocer Python ni scikit-learn. Pydantic valida el contrato de datos en la frontera del sistema.

Aplicación en el proyecto. `main_api.py` con FastAPI: `POST /predict` recibe un `JugadorEntrada` en JSON y devuelve un `PrediccionSalida` con la clase y el porcentaje de probabilidad; `index.html` lo consume con `fetch`.

2.4. Conceptos complementarios de despliegue y monitoreo

2.4.1. Despliegue en CRISP-DM

Un modelo no genera valor mientras viva solo en un *notebook*: en memoria RAM se pierde todo (pesos, reglas de los árboles, medianas de imputación) al cerrar el proceso. El despliegue es la fase que convierte el modelo en un servicio estable al que acceden los usuarios del negocio.

2.4.2. La «regla de oro»: guardar los transformadores junto al modelo

Los datos nuevos deben recibir **exactamente el mismo preprocesamiento** que los datos de entrenamiento. Por eso el paquete `.joblib` no guarda solo el modelo, sino también `columnas_X`, `medianas_estatura`, `mediana_global_estatura`, `mediana_global_edad` y `umbral_elite_eur`. En este proyecto la regla se llevó también al código: la API, el script de

inferencia, el entrenamiento y los experimentos llaman a las mismas funciones de `src/preprocessing/inferencia.py`.

Al aplicar esta regla se detectó un defecto sutil en el script de inferencia de referencia: llamar a `get_dummies(drop_first=True)` sobre un lote nuevo elimina la primera posición *presente en ese lote*, no la categoría base del entrenamiento. En el lote de ejemplo (Defensa, Portero, Mediocampista) el Candidato A (Defensa) quedaba con todas sus columnas en cero, es decir, codificado como Delantero; y una petición individual de un portero perdía su única columna. La versión del proyecto codifica sin `drop_first` y luego alinea con `reindex(columns=columnas_X, fill_value=0)`, lo que garantiza columnas idénticas; el módulo incluye una autoverificación de este caso.

2.4.3. Data Drift y Concept Drift

Data Drift (deriva de datos) ocurre cuando cambia con el tiempo la distribución de las variables de entrada X; por ejemplo, la inflación del mercado de fichajes desplaza todos los valores hacia arriba y el umbral Q3 calculado hoy deja de representar el Top 25 % de mañana. **Concept Drift** (deriva de concepto) ocurre cuando cambia la relación entre X y la variable objetivo Y; por ejemplo, cambios reglamentarios o tácticos que vuelven más o menos valiosas ciertas características físicas (Gama et al., 2014). Ambos degradan silenciosamente un modelo desplegado, por lo que exigen monitoreo y reentrenamiento periódico (§5.4).

3. Análisis de resultados del código y experimentos

3.1. Descripción de los datos y del preprocesamiento

La consulta SQL oficial filtra los jugadores sin valor de mercado o sin fecha de nacimiento, de modo que de los 50 149 registros de `players` quedaron **41 495** jugadores. La edad se calcula en MySQL con `TIMESTAMPDIFF(YEAR, date_of_birth, CURDATE())`. La Tabla 2 resume el estado de los datos antes y después del preprocesamiento.

Tabla 2

Resumen de los datos y del preprocesamiento

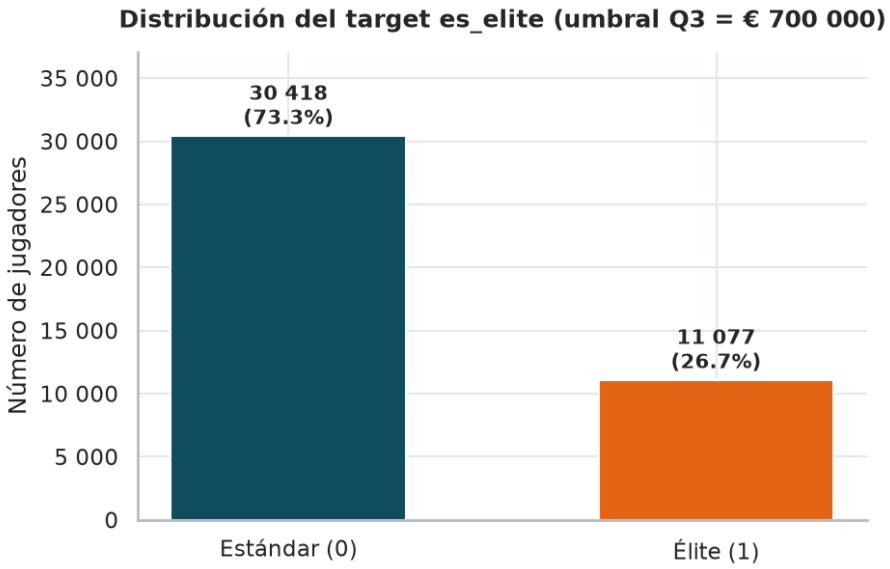
Indicador	Valor
Registros extraídos de MySQL	41 495
Estaturas nulas antes de imputar	2 183 (5.26%)
Edades nulas antes de imputar	0
Nulos después de imputar (estatura / edad)	0 / 0
Distribución de posiciones	Defensa: 13 403; Mediocampista: 12 002; Delantero: 11 361; Portero: 4 602; Sin posición registrada: 127
Umbral Élite (Q3 del valor de mercado)	€ 700 000
Valor de mercado mediano	€ 275 000
Jugadores Élite / Estándar	11 077 (26.69 %) / 30 418 (73.31 %)
Partición estratificada Train / Test	33 196 / 8 299 (Élite: 26.696 % / 26.690 %)
Variables del modelo (X)	`edad`, `estatura_cm`, `posicion_Defender`, `posicion_Goalkeeper`, `posicion_Midfield`, `posicion_Missing`

Nota. Elaboración propia.

La proporción de Élite (26.69 %) es ligeramente superior al 25 % teórico porque muchos jugadores comparten exactamente el valor del umbral (€ 700 000, una cifra redonda típica de Transfermarkt) y la regla `>=` los incluye a todos. El problema es, en todo caso, claramente **desbalanceado** (aproximadamente 73/27), como muestra la Figura 2.

Figura 2

Distribución de la variable objetivo es_elite

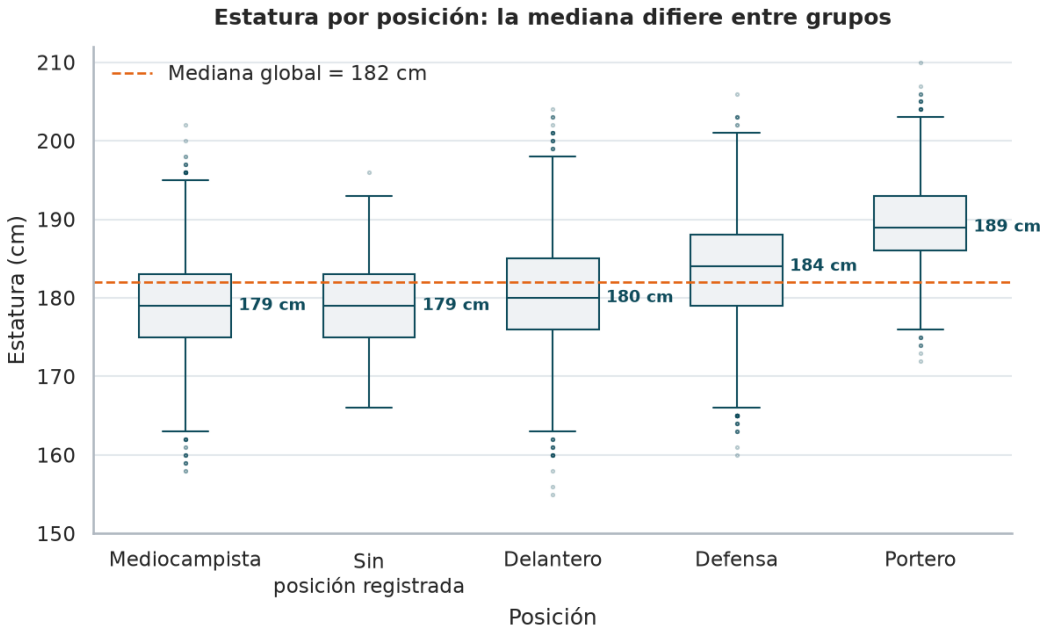


Nota. Elaboración propia.

La Figura 3 justifica la **imputación por mediana agrupada**: la mediana de estatura de los porteros (189 cm) supera en 10 cm a la de los mediocampistas (179 cm), y la de los defensas es 184 cm. Imputar con la mediana global (182 cm) habría asignado a un portero sin dato una estatura de jugador de campo. La edad, sin nulos tras el filtro SQL, se imputa por seguridad con la mediana global (29 años) para soportar datos nuevos incompletos.

Figura 3

Distribución de la estatura por posición



Nota. Elaboración propia. El eje se limita a 150-212 cm; el dataset contiene 11 estaturas físicamente imposibles (< 140 cm), errores de captura que no se alteraron para conservar el preprocesamiento oficial.

Prevención de fuga de datos. En los experimentos, las medianas se calcularon **solo con el conjunto de entrenamiento** y se aplicaron después al de prueba; si se calcularan con todo el dataset, información del test se filtraría al entrenamiento y las métricas serían optimistas. En el modelo final de producción (`exportar_modelo.py`) sí se usan todos los datos, porque ya no hay un conjunto de prueba que proteger.

3.1.1. Protocolo experimental

Todos los modelos se entrenaron con el mismo conjunto de entrenamiento (33 196 jugadores) y se evaluaron con el mismo conjunto de prueba (8 299 jugadores) que el modelo nunca vio, con semilla fija (`random_state=42`) para garantizar la reproducibilidad. Las métricas de la clase Élite (Precision, Recall, F1) se reportan porque es la clase de interés para el negocio. El script completo es `experimentos_modelado.py` y su salida queda en `reports/resultados/`.

3.2. Paradoja de la exactitud

Un clasificador trivial que **siempre** responde «Estándar» (`DummyClassifier(strategy='most_frequent')`) obtiene una exactitud de **73.31 %** en Test,

aunque su Recall para la clase Élite es **0.0**: no identifica ni a uno solo de los 2 215 jugadores Élite del conjunto de prueba. Cualquier modelo debe superar esta referencia en las métricas de la clase minoritaria, no en exactitud.

El experimento reveló que el **árbol podado sin balanceo cae exactamente en esta paradoja**: su accuracy en Test (73.31 %) es la más alta de todos los modelos y su brecha es nula, pero su Recall Élite es 0.0. Con solo cuatro niveles de profundidad, en ninguna de sus 16 hojas los jugadores Élite llegan a ser mayoría, así que todas las hojas votan «Estándar». Juzgado solo por la exactitud parecería el mejor modelo; en realidad es inútil para el scouting.

3.3. Diagnóstico de overfitting: árbol no restringido vs. árbol podado

Tabla 3

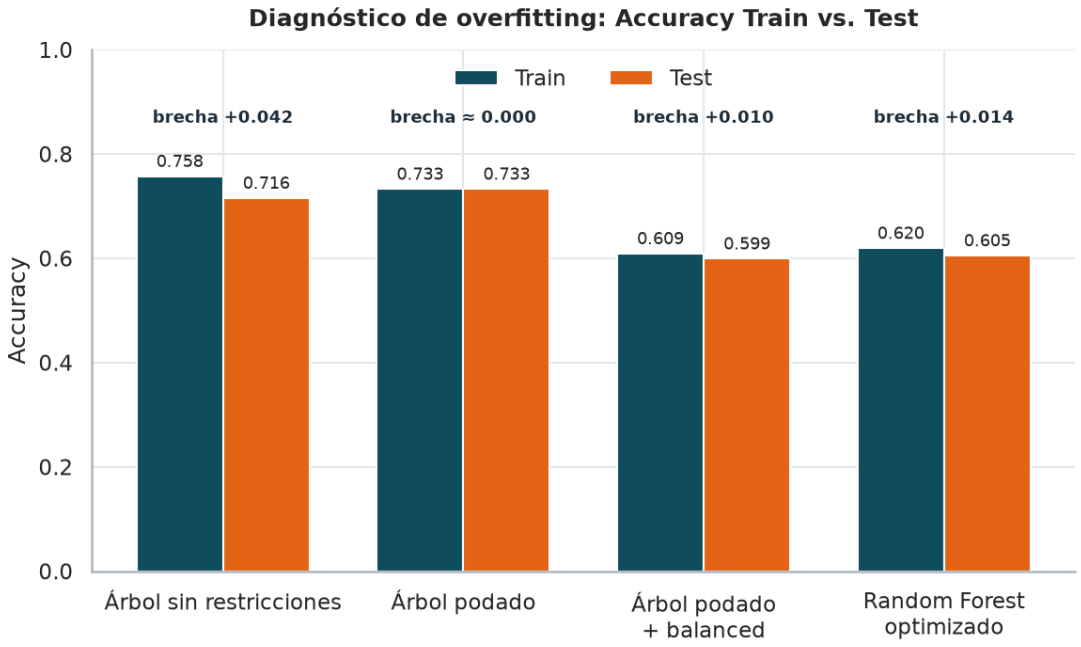
Comparación Train vs. Test y métricas de la clase Élite en el conjunto de prueba

Modelo	Acc. Train	Acc. Test	Brecha	Precision Élite	Recall Élite	F1 Élite	AUC
Árbol sin restricciones	0.7575	0.7157	+0.0418	0.4011	0.1318	0.1984	0.6523
Árbol podado	0.7330	0.7331	-0.0001	0.0000	0.0000	0.0000	0.6785
Árbol podado + balanced	0.6094	0.5993	+0.0100	0.3736	0.7409	0.4967	0.6779
Random Forest optimizado	0.6197	0.6054	+0.0143	0.3808	0.7648	0.5085	0.6937
Dummy (most_frequent)	–	0.7331	–	0.0000	0.0000	0.0000	0.5000

Nota. Elaboración propia a partir de `reports/resultados/tabla_overfitting.csv`. Brecha = Accuracy Train − Accuracy Test. Precision, Recall y F1 se calculan para la clase Élite (1).

Figura 4

Accuracy en entrenamiento y prueba por modelo



Nota. Elaboración propia.

Árbol sin restricciones. Creció hasta 25 niveles y 2 693 hojas, y su exactitud cae de 75.75 % en Train a 71.57 % en Test: una brecha de **4.18 puntos porcentuales**, la mayor de todos los modelos. Es la firma del sobreajuste: el árbol creó reglas para combinaciones muy específicas de edad y estatura que no se repiten en datos nuevos; además, su AUC en Test (0.6523) es el más bajo de los modelos entrenados.

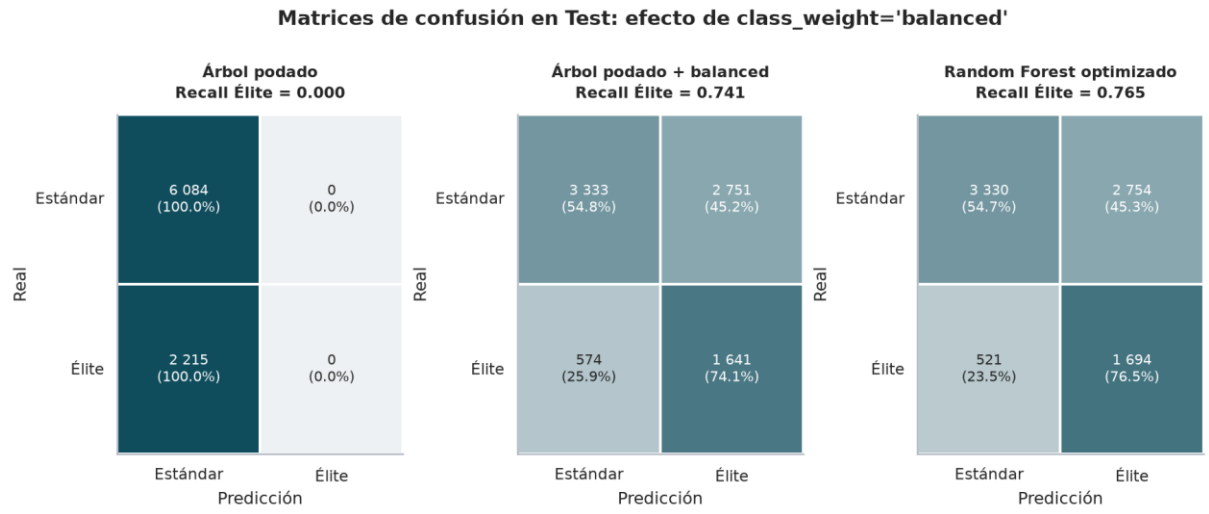
Que la exactitud de entrenamiento del árbol libre sea solo 75.75 % —y no cercana al 100 % como suele ocurrir— tiene una explicación estructural: con tres variables, los 41 495 jugadores se reparten en apenas 3 867 combinaciones distintas de edad, estatura y posición. Muchos jugadores idénticos para el modelo tienen etiquetas distintas, y ningún árbol puede separarlos: es un límite de información de los datos, no del algoritmo.

Árbol podado. La poda previa (`max_depth=4`, `min_samples_leaf=20`) eliminó el sobreajuste (brecha -0.0001) y elevó el AUC a 0.6785, lo que confirma que generaliza mejor al ordenar a los jugadores por probabilidad. Sin embargo, como se vio en §3.2, su umbral de decisión por defecto (50 %) lo lleva a predecir siempre Estándar. La poda resuelve la varianza, pero no el desbalance.

Efecto de `class_weight='balanced'`. Al penalizar más los errores sobre la clase Élite, el mismo árbol podado pasa de Recall 0.000 a **0.741** y su F1 sube de 0 a 0.497. La exactitud baja a 59.93 %, un costo esperado y aceptable: el modelo ahora acepta falsos positivos (jugadores Estándar señalados como Élite) a cambio de no dejar escapar a los Élite. La Figura 5 lo muestra en las matrices de confusión.

Figura 5

Matrices de confusión en Test: árbol podado, árbol podado balanceado y Random Forest



Nota. Elaboración propia.

En términos de negocio, de los 2 215 jugadores Élite del conjunto de prueba, el Random Forest detecta 1 694 (76.5 %) y deja escapar 521. A cambio, marca como candidatos a 2 754 jugadores Estándar, por lo que su precisión es 38.1 %: aproximadamente 4 de cada 10 jugadores que el sistema recomienda observar son realmente Élite, frente a 26.7 % si se eligieran al azar.

3.4. Evaluación del ensamble y curva ROC-AUC

El Random Forest se optimizó con `GridSearchCV` sobre una malla de 72 combinaciones (`n_estimators` [100, 200, 300], `max_depth` [4, 6, 8, None], `min_samples_leaf` [1, 5, 10], `max_features` [sqrt, None]) con validación cruzada estratificada de 5 particiones (360 entrenamientos en 5.6 minutos), optimizando ROC-AUC y registrando también F1. No fue necesario reducir la malla.

Tabla 4

Las cinco mejores combinaciones de hiperparámetros del GridSearchCV

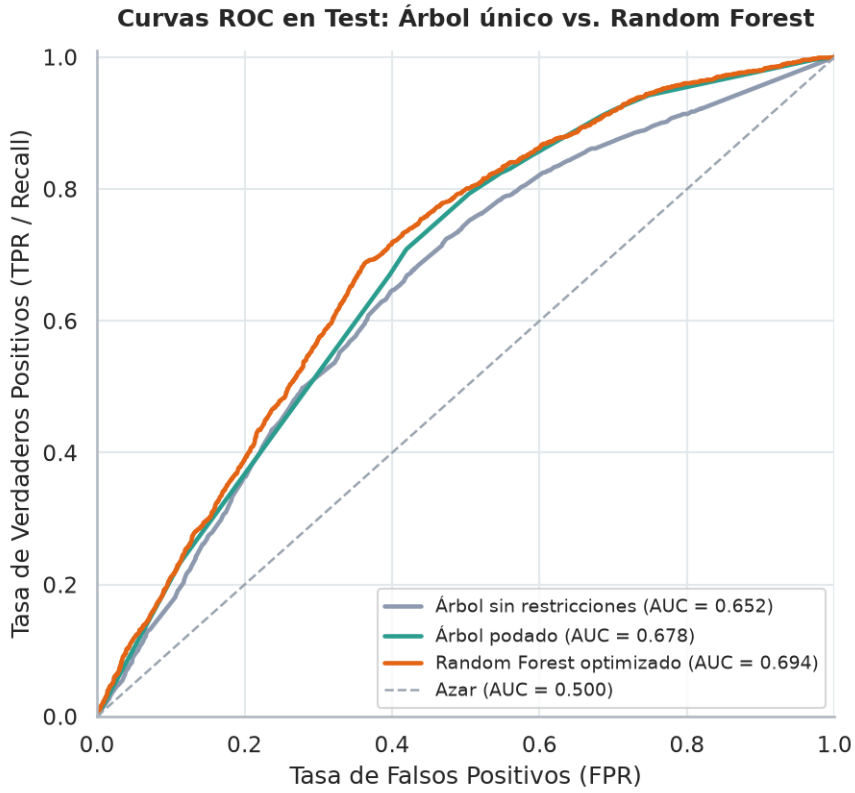
Rango	n_estimators	max_depth	min_samples_leaf	max_features	AUC CV (media ± desv.)	F1 CV
1	300	8	5	sqrt	0.7024 ± 0.0082	0.5114
2	300	8	10	sqrt	0.7024 ± 0.0085	0.5117
3	200	8	5	sqrt	0.7023 ± 0.0082	0.5115
4	200	8	10	sqrt	0.7023 ± 0.0085	0.5113
5	300	8	1	sqrt	0.7021 ± 0.0083	0.5118

Nota. Elaboración propia a partir de `reports/resultados/gridsearch_resultados.csv` (top 10 completo).

La mejor configuración fue **`n_estimators=300`, `max_depth=8`, `min_samples_leaf=5`, `max_features='sqrt'`**, con un ROC-AUC de validación cruzada de **0.7024 ± 0.0082** y F1 de 0.5114. Dos observaciones: (1) las diez mejores combinaciones están separadas por menos de una milésima de AUC, es decir, el modelo es poco sensible a los hiperparámetros dentro de un rango razonable; (2) todas usan `max_features='sqrt'`, lo que confirma el valor del *feature subspacing* para descorrelacionar los árboles. El AUC obtenido en Test (0.6937) es coherente con el de validación cruzada, señal de que la búsqueda no sobreajustó los hiperparámetros.

Figura 6

Curvas ROC comparativas en Test: árbol único vs. Random Forest



Nota. Elaboración propia.

Interpretación del AUC. El Random Forest optimizado alcanza **AUC = 0.6937**, frente a 0.6523 del árbol sin restricciones y 0.6785 del árbol podado. Esto significa que, si se toma al azar un jugador Élite y uno Estándar, el modelo asigna mayor probabilidad de Élite al primero en el 69.4 % de los casos (frente al 50 % del azar). La curva naranja domina a las demás en casi todo el rango de FPR, en especial en la zona de 0.2 a 0.5, que es la región operativa de un filtro de scouting. La mejora del ensamble sobre el árbol único (+0.0414 frente al libre y +0.0153 frente al podado) es consistente con la teoría del *bagging*: promediar 300 árboles reduce la varianza sin aumentar el sesgo.

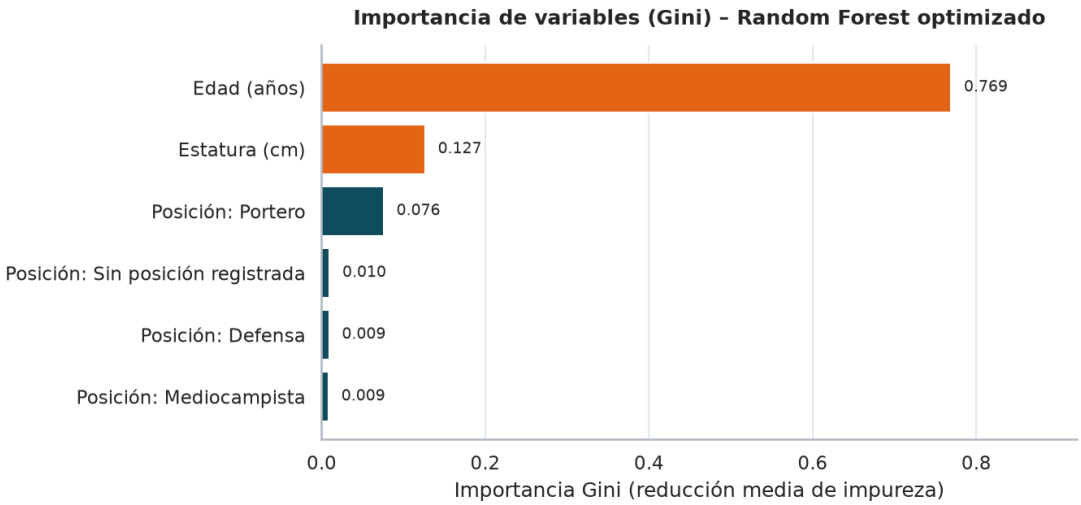
Con honestidad, un AUC cercano a 0.70 indica una capacidad de discriminación **moderada**: útil como primer filtro para priorizar candidatos, pero insuficiente para decidir un fichaje por sí sola. El techo no lo impone el algoritmo, sino la información disponible: tres variables físicas y demográficas no pueden capturar el rendimiento deportivo, la liga o el club, que son los

principales determinantes del valor de mercado. Este resultado delimita el alcance del sistema (§5).

3.5. Importancia de variables: ¿qué variables pesan más y por qué?

Figura 7

Importancia de variables (Gini) del Random Forest optimizado

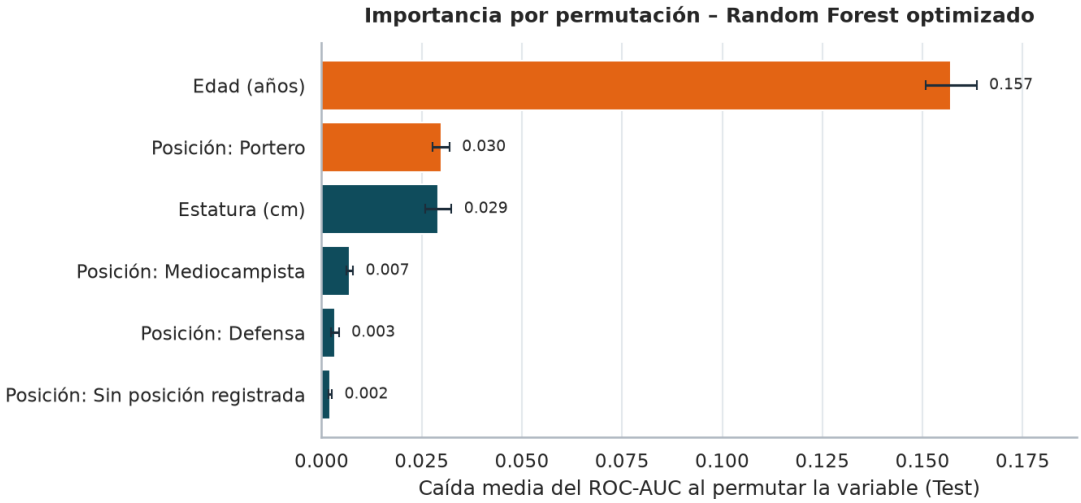


Nota. Elaboración propia.

La Figura 7 muestra una jerarquía muy marcada: **la edad concentra 76.9 % de la importancia Gini**, seguida por la estatura (12.7 %) y la condición de portero (7.6 %); las demás posiciones aportan cerca de 1 % cada una. La importancia por permutación (Figura 8) confirma las mismas tres variables en el top, aunque intercambia el segundo y el tercer lugar: permutar la edad reduce el AUC en 0.157, mientras que la posición de portero (0.030) y la estatura (0.029) pesan prácticamente lo mismo.

Figura 8

Importancia por permutación (caída del ROC-AUC en Test) del Random Forest optimizado



Nota. Elaboración propia. Barras de error: desviación estándar en 10 repeticiones.

Tabla 5

Proporción de jugadores Élite por rango de edad

Rango de edad (años)	Jugadores	Tasa de Élite
≤20	1 987	35.9 %
21-23	5 525	35.1 %
24-26	6 892	38.3 %
27-29	6 937	37.0 %
30-32	6 349	26.7 %
33-35	5 095	16.5 %
≥36	8 710	7.8 %

Nota. Elaboración propia a partir de `metricas.json`. Edad calculada a la fecha de ejecución (noviembre de 2026).

¿Por qué la edad domina? La Tabla 5 muestra la relación que el modelo aprendió: la tasa de Élite se mantiene entre 35.1 % y 38.3 % hasta los 29 años (máximo en 24-26 años) y a partir de ahí se desploma: 26.7 % entre 30 y 32 años, 16.5 % entre 33 y 35 y solo 7.8 % desde los 36. Es la conocida **curva de valor por edad** del mercado de fichajes: los clubes pagan por los jóvenes su potencial y su valor de reventa, el precio se sostiene mientras el jugador está en su plenitud física y cae a partir de la treintena, cuando se descuentan los pocos años de carrera restantes. La relación no es lineal (una meseta seguida de una caída), justo el tipo de patrón que un árbol captura con un par de cortes y que explica la gran reducción de impureza que aporta la edad.

Hay además un efecto del propio dataset que amplifica el peso de la edad: **8 710 jugadores (21.0% del total) tienen 36 años o más**, una proporción muy superior a la de futbolistas activos de esa edad. La tabla `players` conserva jugadores de temporadas antiguas cuyo último valor de mercado quedó congelado al retirarse o dejar las ligas cubiertas; como la edad se calcula a la fecha actual, aparecen como veteranos con valores bajos. Parte del poder predictivo de la edad consiste, por tanto, en reconocer jugadores inactivos, algo que debería corregirse filtrando por jugadores activos (§5.4).

¿Por qué importa la posición de portero y no las demás? La tasa de Élite de los porteros es 15.3 %, frente a 29.6 % de los delanteros, 28.2 % de los mediocampistas y 27.0 % de los defensas. El mercado cotiza menos a los porteros: cada equipo alinea uno solo, su influencia en goles y asistencias —las estadísticas que más mueven el precio— es indirecta, y los traspasos millonarios de porteros son excepcionales; en cambio, delanteros y mediocampistas ofensivos concentran las mayores cotizaciones. Las tres posiciones de campo tienen tasas parecidas entre sí, por eso sus columnas *dummy* aportan poco: la única distinción posicional realmente informativa es «portero frente a jugador de campo».

¿Y la estatura? Su importancia Gini (segunda) es mayor que la de permutación (tercera, empatada con portero). Hay dos razones. Primera, un sesgo conocido: la importancia Gini favorece a las variables continuas, que ofrecen muchos puntos de corte posibles, frente a las binarias, que ofrecen uno solo (Strobl et al., 2007); por eso la permutación, que mide el efecto real sobre el AUC en datos no vistos, es la referencia más fiable. Segunda, la estatura contiene información en parte redundante con la posición (los porteros son más altos), de modo que el bosque puede usarla como sustituto de la posición. En el fútbol la estatura es un requisito en ciertos roles (porteros y defensas centrales), pero no un generador de valor por sí misma: la técnica, la velocidad y el rendimiento —que no están en el modelo— pesan mucho más.

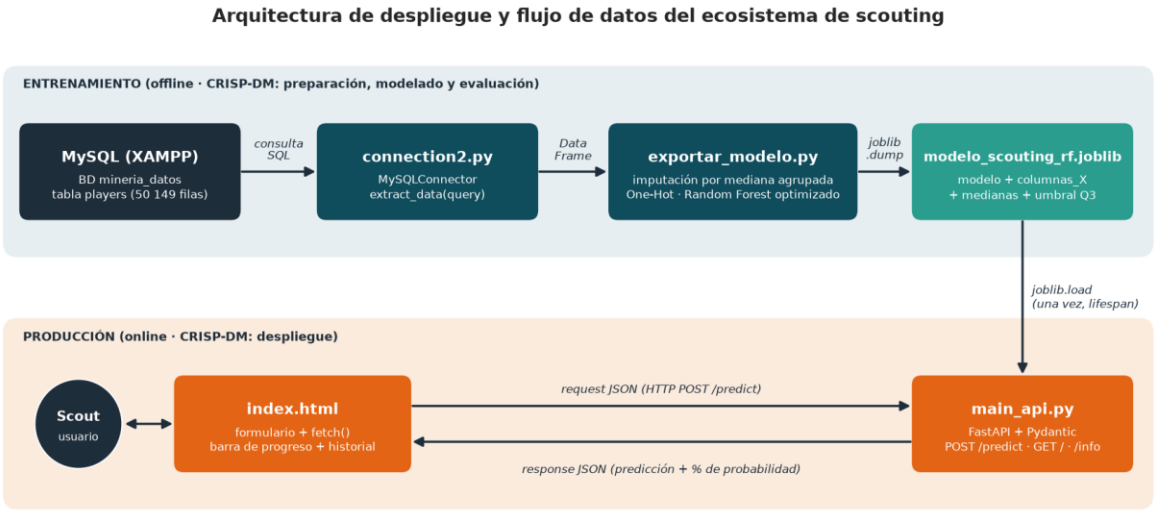
Limitaciones de un modelo con tres variables. El valor de mercado depende sobre todo de variables ausentes: goles, asistencias y minutos jugados, la liga y el club donde compite el jugador, su nacionalidad y selección, la duración de su contrato y su historial de lesiones. Por eso el modelo debe entenderse como un **filtro demográfico inicial**, no como un estimador del talento.

4. Arquitectura de despliegue

La Figura 9 muestra el flujo completo de datos. En la etapa de **entrenamiento** (offline), `connection2.py` ejecuta la consulta SQL en MySQL (XAMPP) y entrega un DataFrame; `exportar_modelo.py` imputa, codifica, entrena el Random Forest con los mejores hiperparámetros y serializa un paquete con el modelo y sus transformadores. En la etapa de **producción** (online), `main_api.py` carga ese paquete una sola vez al iniciar y atiende peticiones HTTP; `index.html` envía los datos del jugador con `fetch` y muestra la respuesta al scout.

Figura 9

Arquitectura de despliegue y flujo de datos



Nota. Elaboración propia.

4.1. Exportación del modelo

El modelo final se reentrena con el 100 % de los datos (41 495 jugadores). Es correcto porque los hiperparámetros ya se eligieron con validación cruzada sobre el conjunto de entrenamiento y el desempeño ya se estimó en el conjunto de prueba; usar todos los registros solo le da al modelo de producción más información. El paquete conserva las claves del script de referencia (`modelo`, `columnas_X`, `medianas_estatura`, `mediana_global_estatura`, `mediana_global_edad`, `umbral_elite_eur`) y agrega metadatos: `posiciones_validas`, `hiperparametros`, `metricas_test`, `fecha_entrenamiento`, `version_sklearn` y `n_registros`. El script verifica al final que el archivo recargado con `joblib.load` produce las mismas predicciones.

4.2. Contrato JSON de la API

La API expone cuatro *endpoints*: `GET /` (estado de la API y del modelo), `GET /info` (metadatos), **`POST /predict`** (un jugador) y `POST /predict/lote` (lista de jugadores). Pydantic valida la entrada: la posición debe ser uno de los valores del entrenamiento (`Attack`, `Defender`, `Goalkeeper`, `Midfield`, `Missing`), la edad un entero entre 15 y 45 y la estatura un número entre 150 y 215 o `null`; cualquier violación devuelve un error **422** antes de llegar al modelo. Ejemplo de petición y respuesta reales:

```
POST http://127.0.0.1:8000/predict
Content-Type: application/json

{
  "jugador": "Candidato B",
  "posicion": "Goalkeeper",
  "edad": 29,
  "estatura_cm": null
}
```

```
HTTP/1.1 200 OK

{
  "jugador": "Candidato B",
  "prediccion": "Estándar",
  "clase": 0,
  "probabilidad_elite": 33.88,
  "probabilidad_estandar": 66.12,
  "estatura_utilizada": 189.0,
  "estatura_imputada": true,
  "umbral_elite_eur": 700000.0
}
```

Si la estatura llega como `null`, la API la imputa con la mediana de la posición guardada en el paquete y lo informa con `estatura_imputada: true`. El conjunto de pruebas automatizadas `tests/test_api.py` (pytest + `TestClient`) verifica el health check, una predicción normal, la imputación de estatura nula, el rechazo de una posición inválida y de edades fuera de rango (422), y el endpoint por lotes.

4.3. API en ejecución

Figura 10

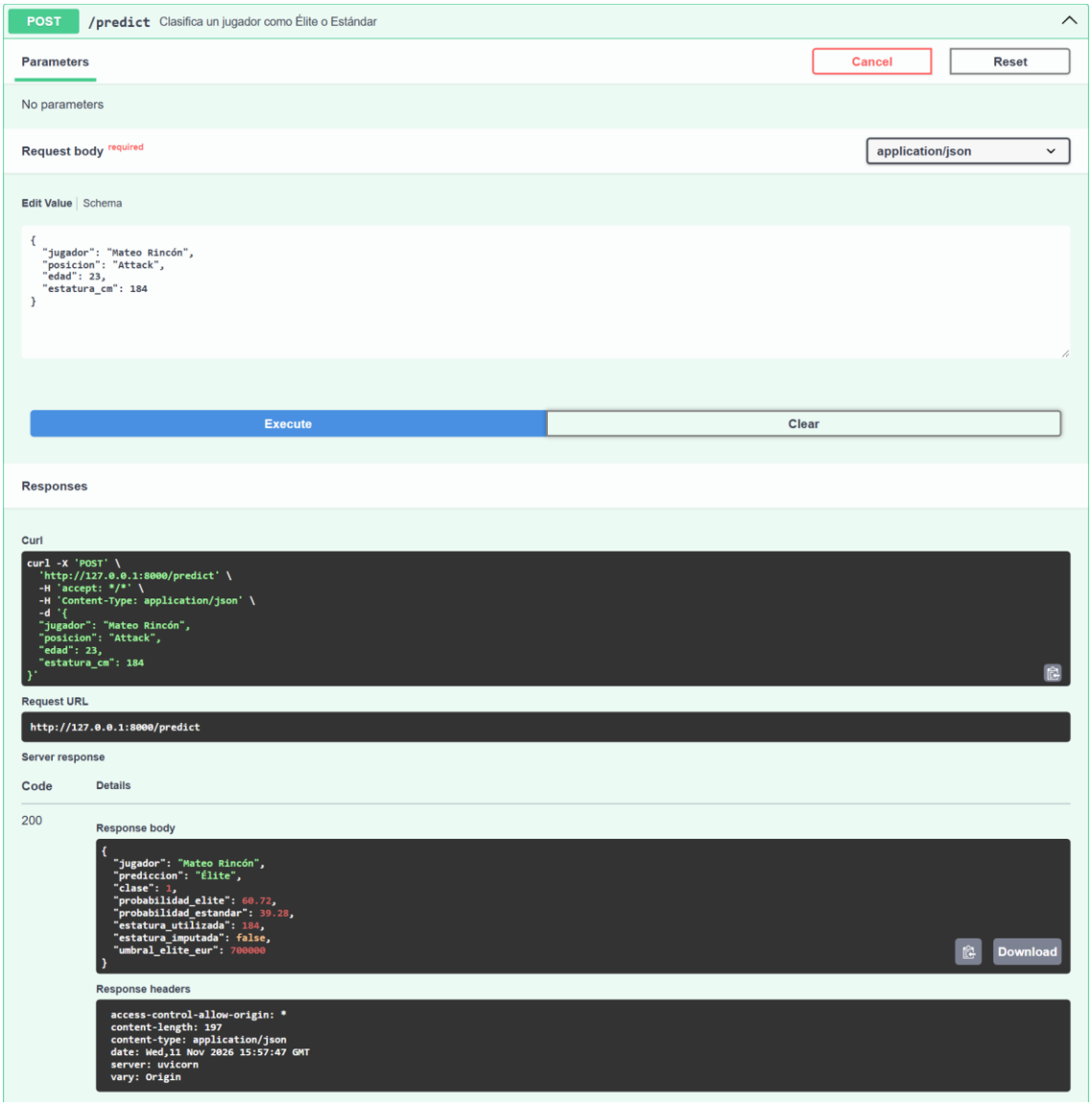
Documentación interactiva Swagger UI generada por FastAPI (/docs)



Nota. Captura propia de la API en ejecución (`uvicorn main_api:app --port 8000`).

Figura 11

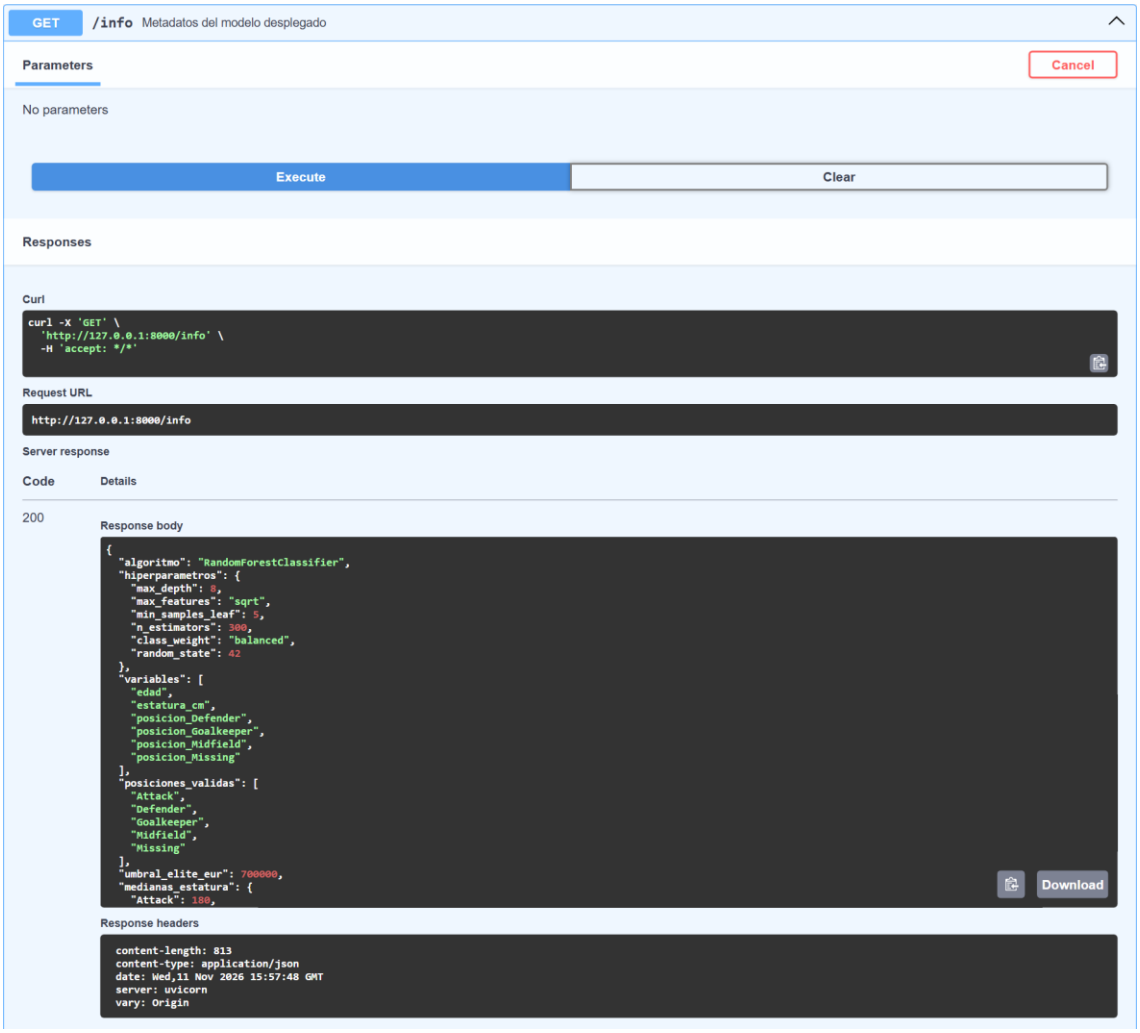
Ejecución de POST /predict desde Swagger con un jugador nuevo y respuesta 200



Nota. Captura propia.

Figura 12

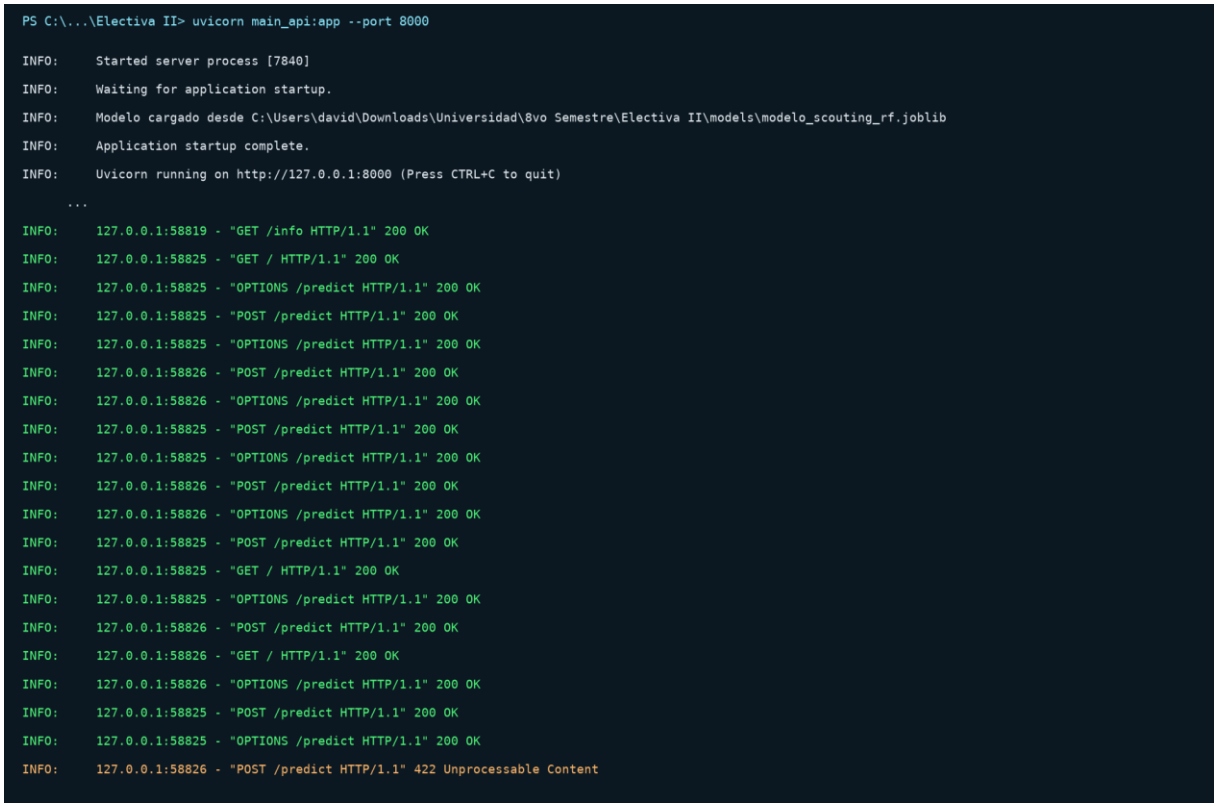
Respuesta del endpoint GET /info con los metadatos del modelo desplegado



Nota. Captura propia.

Figura 13

Registro de la consola de uvicorn con las peticiones atendidas durante las pruebas



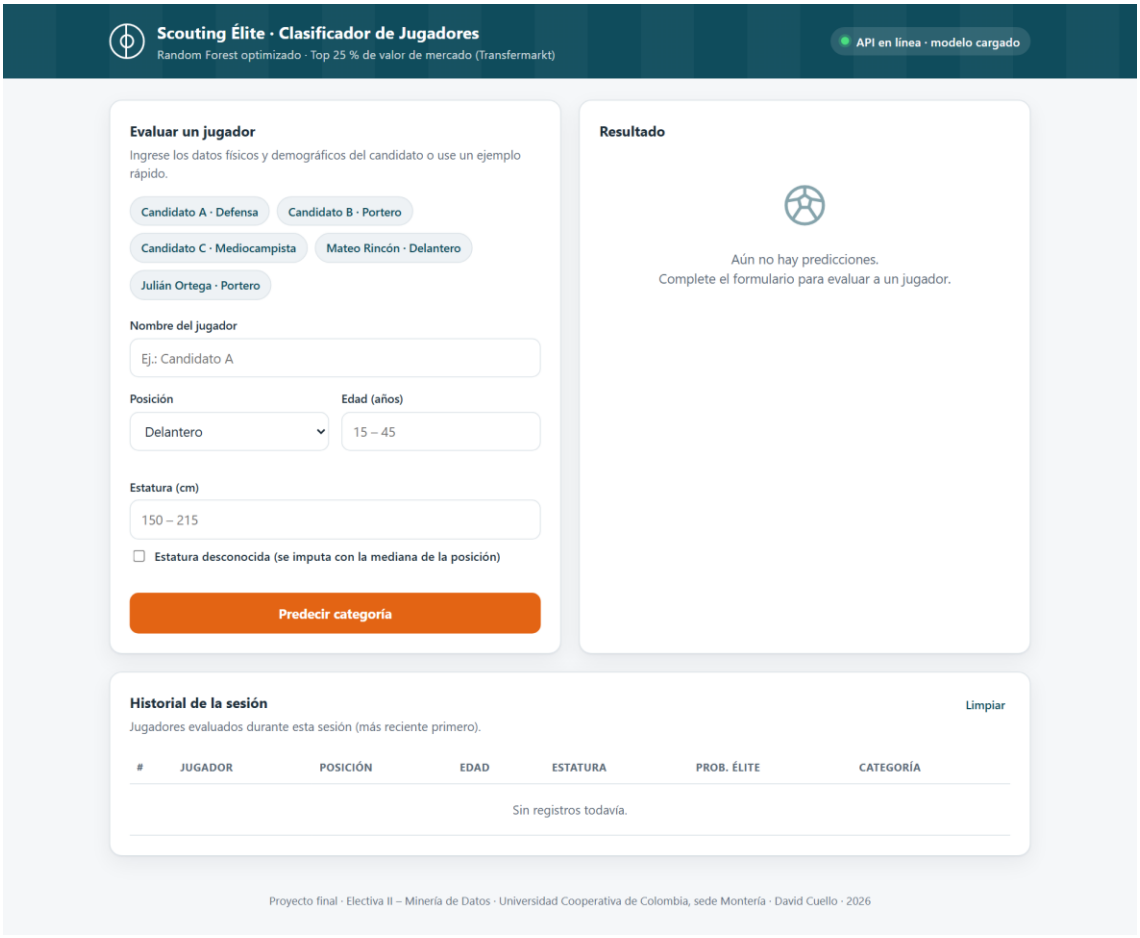
Nota. Captura propia del log del servidor (respuestas 200 en verde; validación 422 en naranja).

4.4. Aplicativo web respondiendo a jugadores nuevos

`index.html` es un único archivo con HTML, CSS y JavaScript sin dependencias externas, de modo que funciona sin conexión a internet. Al cargar consulta `GET /` y muestra si la API está en línea; el formulario traduce las posiciones al español, permite marcar «Estatura desconocida» (envía `null`), valida los rangos en el cliente y ofrece botones de ejemplo rápido con los candidatos A, B y C del docente y dos jugadores adicionales. El resultado se muestra con una insignia Élite/Estándar y una barra de progreso animada cuyo color cambia con la probabilidad; cada evaluación se agrega al historial de la sesión. Los errores 422 se traducen a mensajes comprensibles y, si la API está apagada, se indica cómo iniciarla.

Figura 14

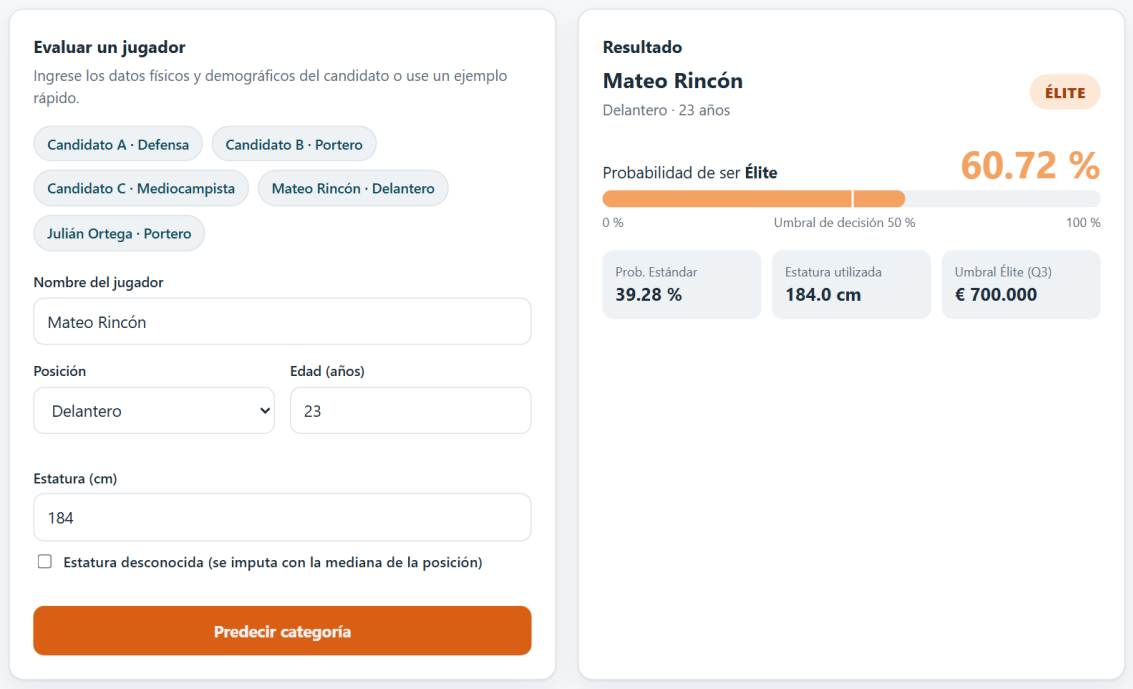
Aplicativo web al iniciar, con la API en línea



Nota. Captura propia (Chromium).

Figura 15

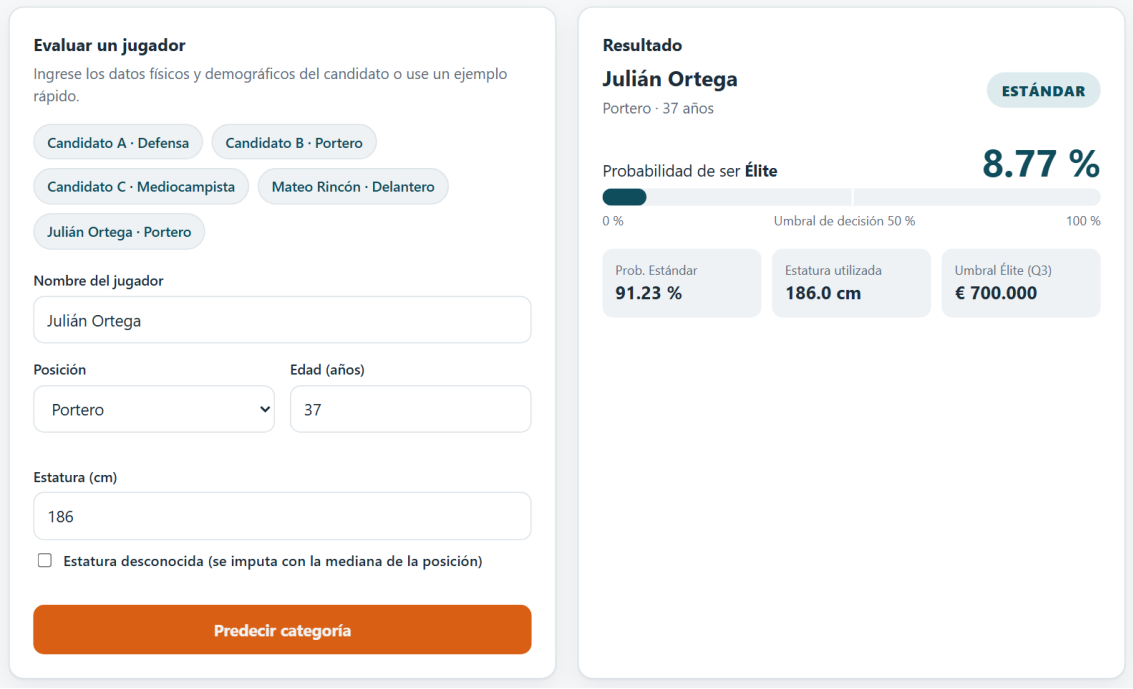
Predicción de un jugador nuevo clasificado como Élite



Nota. Captura propia.

Figura 16

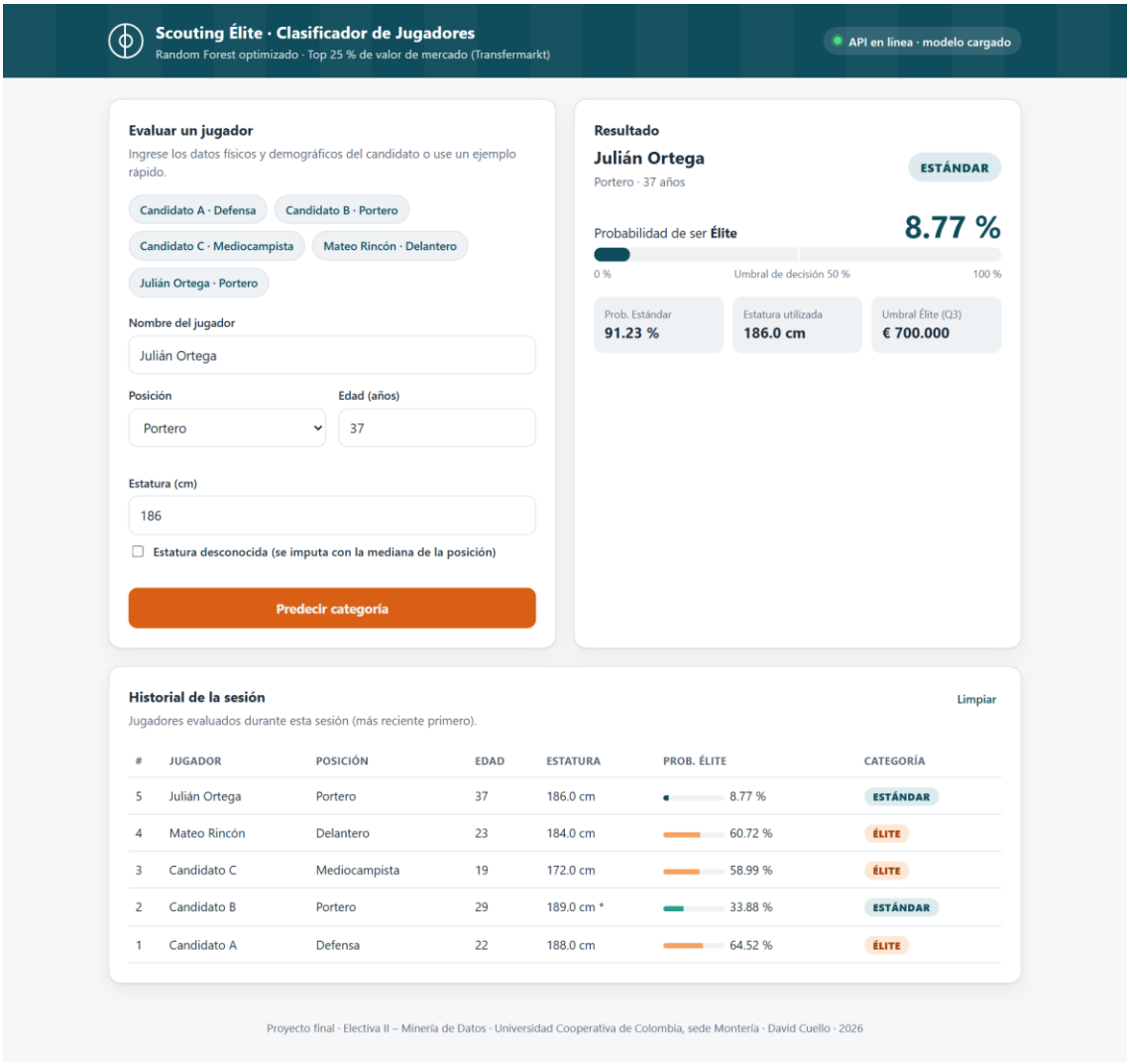
Predicción de un jugador nuevo clasificado como Estándar



Nota. Captura propia.

Figura 17

Historial de la sesión con cinco jugadores nuevos evaluados



Nota. Captura propia. El asterisco junto a la estatura indica que fue imputada.

5. Conclusiones y recomendaciones de negocio

5.1. Conclusiones

- Se construyó y desplegó un ecosistema analítico completo: MySQL → Python → modelo serializado → API REST → aplicativo web, cubriendo las seis fases de CRISP-DM. El sistema responde en tiempo real a jugadores nuevos y está respaldado por pruebas automatizadas.
- El árbol de decisión sin restricciones evidenció sobreajuste (brecha de 4.18 puntos y el AUC más bajo, 0.652); la poda previa eliminó la brecha y mejoró el AUC.
- La exactitud resultó una métrica engañosa: el modelo trivial y el árbol podado sin balanceo superan el 73 % de accuracy sin detectar a un solo jugador Élite. `class_weight='balanced'` fue decisivo para que los modelos fueran útiles (Recall Élite de 0 a 0.765 en el Random Forest).
- El Random Forest optimizado con GridSearchCV y K-Fold estratificado es el mejor modelo (AUC = 0.6937, F1 Élite = 0.5085, brecha de solo 1.43 puntos), lo que confirma la ventaja del ensamble sobre el árbol único.
- La edad es, con diferencia, la variable más influyente, coherente con la curva de valor por edad del mercado; ser portero reduce la probabilidad de ser Élite y la estatura aporta poco por sí sola.

5.2. Cómo apoya el sistema la toma de decisiones en scouting

- **Filtro inicial de candidatos.** Ante una lista larga de perfiles, el sistema descarta rápidamente a los de baja probabilidad y conserva al 76 % de los jugadores Élite, con una tasa de acierto entre los recomendados de 38 %, muy superior a la tasa base del 27 %.
- **Priorización de visorías.** La probabilidad (y no solo la etiqueta) permite ordenar a los candidatos y asignar primero los viajes y las horas de análisis a quienes tienen mayor potencial de mercado.
- **Ahorro de costos y decisiones trazables.** Cada recomendación queda registrada con datos y criterios explícitos, lo que reduce el sesgo subjetivo y facilita justificar las decisiones ante la dirección deportiva.

- **Integración.** Al exponerse como API REST, el mismo modelo puede consumirse desde el aplicativo web, una hoja de cálculo, una aplicación móvil o el sistema de gestión deportiva del club, y el endpoint por lotes permite evaluar listas completas de una sola vez.

5.3. Limitaciones

- **Pocas variables.** Edad, estatura y posición explican solo una parte del valor de mercado; el AUC moderado refleja ese techo de información.
- **Sesgo del valor de mercado como target.** El valor de Transfermarkt es una estimación colectiva que mezcla talento con factores comerciales (liga, club, nacionalidad, contrato). El modelo predice valor de mercado, no calidad deportiva.
- **Temporalidad e inflación.** Las valoraciones de distintas épocas no son comparables y la edad se calcula a la fecha actual, también para jugadores con valoraciones antiguas.
- **Calidad de datos.** Existen estaturas físicamente imposibles y una categoría de posición «Missing» que deberían depurarse en la fuente.

5.4. Recomendaciones y trabajo futuro

- **Más variables de rendimiento**: integrar las tablas `appearances`, `games`, `clubs` y `competitions` del mismo dataset (goles, asistencias, minutos, nivel de la liga y del club) y el pie hábil del jugador.
- **Filtrar jugadores activos y valoraciones recientes** (tabla `player_valuations`) para que la edad y el valor correspondan al mismo momento.
- **Reentrenamiento periódico** (por ejemplo, después de cada ventana de fichajes) recalculando el umbral Q3, y **monitoreo de Data Drift y Concept Drift**: comparar la distribución de las entradas y el desempeño del modelo frente a las valoraciones reales para detectar degradación.
- **Ajuste del umbral de decisión** según el costo de negocio: un club que prefiera no perder talento puede bajar el umbral (más recall); uno con poco presupuesto de visoría puede subirlo (más precisión).

- **Seguridad de la API** antes de exponerla fuera de la red local: autenticación (API key u OAuth2), CORS restringido a los dominios del club, HTTPS, límites de peticiones y registro de auditoría.
- **Explicabilidad individual** (por ejemplo, valores SHAP) para mostrar al scout por qué el modelo calificó así a cada jugador, y empaquetado con Docker para un despliegue reproducible.

6. Referencias

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Wadsworth.

Cariboo, D. (2026). *Football data from Transfermarkt* [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/davidcariboo/player-scores

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys, 46*(4), Artículo 44. https://doi.org/10.1145/2523813

Joblib developers. (2026). *Joblib: Running Python functions as pipeline jobs* [Documentación de software]. https://joblib.readthedocs.io/

Kohavi, R. (1995). A study of cross-validation and bootstrap for accuracy estimation and model selection. En *Proceedings of the 14th International Joint Conference on Artificial Intelligence* (Vol. 2, pp. 1137–1143). Morgan Kaufmann.

Mejía Villalba, R. (2026). *Actividad final y guía de la última clase: despliegue de modelos* [Material de clase]. Electiva II – Minería de Datos, Universidad Cooperativa de Colombia, sede Montería.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D.,

Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Pydantic. (2026). *Pydantic documentation* (Versión 2) [Documentación de software]. https://docs.pydantic.dev/

Ramírez, S. (2026). *FastAPI* [Documentación de software]. https://fastapi.tiangolo.com/

scikit-learn developers. (2026). *scikit-learn user guide* (Versión 1.9) [Documentación de software]. https://scikit-learn.org/stable/user_guide.html

Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics, 8*, Artículo 25. https://doi.org/10.1186/1471-2105-8-25

7. Anexos

Anexo A. Estructura de carpetas del proyecto

```
Electiva II/
├── src/
│   ├── database/connection2.py       Conector MySQL (mysql-connector-python + .env)
│   ├── preprocessing/inferencia.py   Preprocesamiento compartido ("regla de oro")
│   └── estilo.py                     Paleta y estilo de gráficas
├── experimentos_modelado.py          EDA, overfitting, GridSearchCV, ROC, importancias
├── exportar_modelo.py                Entrenamiento final y serialización (Joblib)
├── inferencia_nuevos_jugadores.py    Simulación de inferencia (Candidatos A, B, C)
├── main_api.py                       API REST (FastAPI + Pydantic)
├── index.html                        Aplicativo web cliente (HTML/CSS/JS)
├── models/modelo_scouting_rf.joblib  Paquete del modelo exportado
├── tests/test_api.py                 Pruebas automatizadas de la API (pytest)
├── scripts/                          Diagramas, capturas e informe
├── reports/
│   ├── figuras/                      Figuras 01-09
│   ├── capturas/                     Evidencias de la API y del aplicativo web
│   └── resultados/                   metricas.json, tablas CSV, mejores_parametros.json
├── requirements.txt · README.md · .env.example
└── Informe_Final_Scouting_David_Cuello.pdf
```

Anexo B. Instrucciones de ejecución

Requisitos: Python 3.11 o superior, MySQL/MariaDB (XAMPP) con la base `mineria_datos` cargada y un archivo `.env` con las credenciales (ver `.env.example`). En PowerShell, desde la carpeta del proyecto:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt


python experimentos_modelado.py          # 1. Experimentos y evidencias
python exportar_modelo.py                # 2. Exporta el modelo .joblib
python inferencia_nuevos_jugadores.py    # 3. (Opcional) Prueba de inferencia
python -m pytest tests -v                # 4. (Opcional) Pruebas de la API
uvicorn main_api:app --reload --port 8000   # 5. API en http://127.0.0.1:8000/docs
start index.html                         # 6. Abre el aplicativo web
```

Ejemplo de consumo de la API desde PowerShell:

```
$body = @{ jugador="Candidato A"; posicion="Defender"; edad=22; estatura_cm=188 } |
         ConvertTo-Json
Invoke-RestMethod -Uri http://127.0.0.1:8000/predict -Method Post `
                  -Body $body -ContentType "application/json"
```